

Laboratorio Calificado 12

2026-1

ÍNDICE

1	Instrucciones.....	3
2	Contexto.....	3
3	Objetivos.....	4
4	Requisitos.....	4
5	Entregables.....	7
6	Rúbrica de Evaluación.....	7

Laboratorio Calificado 12

Implementación de la operación de mantenimiento preventivo con manejo de transacciones, modelado, GUI y base de datos relacional para la entidad 'Unidad Robótica'

1 Instrucciones

Se ha creado la tarea “Laboratorio Calificado 12” en la sección correspondiente de la página del curso-horario en PAIDEIA. Cada alumno deberá entregar un archivo comprimido (formato ZIP) con los entregables detallados más adelante. El nombre del archivo comprimido deberá tener el siguiente formato:

L12_<Código del alumno de 8 dígitos>.zip

Como ejemplo, el nombre de archivo del alumno 20236969 sería “L12_20236969.zip”. Es completa responsabilidad de cada alumno respetar el formato de entrega para evitar problemas en la calificación.

2 Contexto

A lo largo del curso, los estudiantes han desarrollado un Sistema de Gestión de Ensamblaje en una Línea de Producción Automatizada de Mecatrónica, incorporando:

- **Gestión de entidades críticas de planta:** Unidad Robótica, Operador y Proceso de Manufactura.
- **Evolución de la capa de datos:** Transición desde la persistencia en archivos locales planos hasta una base de datos relacional robusta en SQL Server.
- **Diseño de interfaces profesionales:** Interfaz gráfica de usuario (GUI) interactiva implementada en C++/CLI con Windows Forms.
- **Módulos de simulación matemática:** Análisis visual del espacio de trabajo y zonas límites de seguridad mediante modelados geométricos polares (Cardioide/Límaco).
- **Operaciones transaccionales óptimas:** Operaciones CRUD integradas sobre la entidad UnidadRobotica invocando procedimientos almacenados (*Stored Procedures*).

En esta fase avanzada del ciclo de vida del software, el sistema debe permitir registrar de forma segura las **operaciones de mantenimiento preventivo y correctivo de las unidades robóticas**, las cuales son fundamentales para mitigar fallas catastróficas y garantizar la continuidad operativa en la gestión industrial.

Para este fin, se estructurará una lógica transaccional de tipo **Maestro-Detalle**, donde cada orden de mantenimiento consta de:

- **Una cabecera (MantenimientoCabecera):** Almacena los datos generales del servicio técnico (ID del mantenimiento, fecha de ejecución, ID del robot intervenido y el operador técnico responsable).
- **Uno o varios detalles (MantenimientoDetalle):** Especifica la lista desglosada de las actividades físicas o calibraciones realizadas sobre el brazo mecánico durante dicha parada (e.g., "Lubricación de engranajes en eje base", "Reajuste de sensores de proximidad", "Actualización del firmware del controlador").

Toda la información transaccional y la lógica de negocio serán almacenadas y validadas en **SQL Server** mediante procedimientos almacenados, asegurando que la cabecera y

sus respectivos detalles se registren de forma atómica y consistente en el servidor de la línea de producción.

3 Objetivos

El estudiante deberá desarrollar e integrar las siguientes actividades en su solución arquitectónica:

- **Configuración Relacional (SQL Server):** Ejecutar el script transaccional en el servidor para instanciar las tablas de jerarquía Maestro-Detalle y los procedimientos almacenados que controlarán la atomicidad de los mantenimientos.
- **En la capa del Modelo (Model):** Modelar e implementar las clases orientadas a objetos correspondientes:
 - MantenimientoCabecera (para los metadatos de la orden de servicio del brazo robótico).
 - MantenimientoDetalle (para encapsular de forma independiente cada actividad técnica o calibración ejecutada).
- **En la capa de Persistencia (Persistence):** Implementar los métodos de conectividad en C++/CLI necesarios para invocar procedimientos almacenados de manera segura, parametrizando los valores de ingeniería y asegurando el manejo correcto de **transacciones locales de base de datos** para resguardar la consistencia de los datos de planta ante fallas de red.
- **En la capa de Control (Controller):** Integrar la lógica en el controlador de mantenimientos encargado de coordinar, validar y transferir los objetos del flujo transaccional y su lista dinámica de actividades detalladas hacia el repositorio relacional.
- **En la capa de Presentación (View / GUI):** Diseñar un formulario interactivo Maestro-Detalle en Windows Forms que permita al operario:
 - Seleccionar la **Unidad Robótica** a intervenir e ingresar los datos de cabecera.
 - Añadir dinámicamente múltiples tareas técnicas a una lista temporal en pantalla (mediante una grilla o lista de control).
 - Confirmar el guardado general para procesar y enviar de forma masiva y segura el bloque transaccional directo hacia SQL Server.
-

4 Requisitos

a) Conexión a la base de datos

El estudiante deberá conectarse a:

- Su propia instancia de SQL Server (AWS), o
- La base de datos del curso (credenciales enviadas por correo electrónico).

b) Script inicial (provisto por el docente)

El siguiente script debe ejecutarse en SQL Server:

Sql

```
-- Verificar y eliminar tablas existentes respetando el orden de dependencia (primero el detalle)
IF OBJECT_ID('dbo.MantenimientoDetalle', 'U') IS NOT NULL
    DROP TABLE MantenimientoDetalle;
GO

IF OBJECT_ID('dbo.MantenimientoCabecera', 'U') IS NOT NULL
    DROP TABLE MantenimientoCabecera;
GO

-- =====
-- 1. CREACIÓN DE TABLAS (MAESTRO - DETALLE)
-- =====

-- Tabla Maestro: Cabecera del Mantenimiento
CREATE TABLE MantenimientoCabecera (
    MantenimientoId INT PRIMARY KEY IDENTITY(1,1),
    RobotId INT,
    FechaMantenimiento DATETIME DEFAULT GETDATE(),
    Tecnico NVARCHAR(100),
    TipoMantenimiento NVARCHAR(50), -- e.g., 'Preventivo', 'Correctivo', 'Calibración'
    FOREIGN KEY (RobotId) REFERENCES UnidadRobotica(RobotId)
);
GO

-- Tabla Detalle: Actividades específicas ejecutadas
CREATE TABLE MantenimientoDetalle (
    MantenimientoDetailId INT PRIMARY KEY IDENTITY(1,1),
    MantenimientoId INT,
    Actividad NVARCHAR(200), -- e.g., 'Lubricación de ejes', 'Calibración de home'
    DuracionMinutos INT,
    FOREIGN KEY (MantenimientoId) REFERENCES MantenimientoCabecera(MantenimientoId)
);
GO

-- =====
-- 2. PROCEDIMIENTOS ALMACENADOS (STORED PROCEDURES)
-- =====

-- Procedimiento para registrar la Cabecera (Retorna el ID generado)
IF OBJECT_ID('dbo.usp_CreateMantenimiento', 'P') IS NOT NULL
    DROP PROCEDURE usp_CreateMantenimiento;
GO

CREATE PROCEDURE usp_CreateMantenimiento
    @RobotId INT,
    @Tecnico NVARCHAR(100),
    @TipoMantenimiento NVARCHAR(50),
    @MantenimientoId INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO MantenimientoCabecera (RobotId, Tecnico, TipoMantenimiento)
    VALUES (@RobotId, @Tecnico, @TipoMantenimiento);

    -- Captura el último ID autogenerado por la identidad en esta sesión
    SET @MantenimientoId = SCOPE_IDENTITY();
END;
GO

-- Procedimiento para insertar cada Actividad en el Detalle
IF OBJECT_ID('dbo.usp_AddMantenimientoDetail', 'P') IS NOT NULL
    DROP PROCEDURE usp_AddMantenimientoDetail;
GO
```

```
CREATE PROCEDURE usp_AddMantenimientoDetail
    @MantenimientoId INT,
    @Actividad NVARCHAR(200),
    @DuracionMinutos INT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO MantenimientoDetalle (MantenimientoId, Actividad, DuracionMinutos)
    VALUES (@MantenimientoId, @Actividad, @DuracionMinutos);
END;
GO
```

c) Actividades obligatorias del estudiante

1. En la capa del Modelo (Model) — OBLIGATORIO

Modelado orientado a objetos de las clases de la estructura transaccional:

- **Clase MantenimientoCabecera (Cabecera):**
 - **Atributos mínimos requeridos:**
 - int mantenimientoid;
 - int robotId;
 - String^ tecnico;
 - String^ tipoMantenimiento; (e.g., *Preventivo, Correctivo*)
 - DateTime fechaMantenimiento;
 - List<MantenimientoDetalle^>^ actividades; (*Lista dinámica de detalles asociados*)
 - **Requisitos:** Implementar constructores apropiados (por defecto y con parámetros) y propiedades de acceso encapsuladas.
- **Clase MantenimientoDetalle (Detalle):**
 - **Atributos mínimos requeridos:**
 - int mantenimientoDetailId;
 - String^ actividad; (e.g., *Calibración de home, cambio de grasa*)
 - int duracionMinutos;
 - **Requisitos:** Implementar constructores apropiados para la inicialización de los componentes físicos de la tarea.

2. En la capa de Persistencia (Persistence)

- Implementar los métodos de conectividad con ADO.NET (SqlConnection, SqlCommand) encargados de conectarse con SQL Server para ejecutar de manera segura:
 - usp_CreateMantenimiento (debe capturar y devolver el Mantenimientoid autogenerado como parámetro de salida).
 - usp_AddMantenimientoDetail (invocado de forma iterativa según la cantidad de tareas agregadas).
- **Manejo de Robustez:** Garantizar el control adecuado de excepciones de base de datos (SqlException).
- **Atomicidad:** Validar el ciclo transaccional completo en base de datos para asegurar el guardado íntegro de la orden de planta.

3. En la capa de Control o Servicio (Controller)

- Diseñar un método orquestador central que reciba como argumentos:
 - El objeto contenedor con los datos de la cabecera del mantenimiento.
 - La colección estructurada (List) de las actividades detalladas.
- **Administrar la transacción a alto nivel:**
 - Solicitar el registro de la cabecera en el servidor y recuperar el ID relacional.
 - Recorrer e insertar secuencialmente cada detalle utilizando la llave foránea obtenida.
 - Controlar fallas en el flujo lógico y reportar limpiamente los mensajes de error hacia la interfaz de usuario.

4. En la capa de Presentación (View – C++/CLI)

- Desarrollar e integrar una ventana gráfica interactiva con Windows Forms que permita al operario técnico:
 - Ingresar o seleccionar el ID único de la **Unidad Robótica** a intervenir.
 - Especificar el nombre del técnico encargado, el tipo de mantenimiento y la fecha programada empleando controles de entrada apropiados (TextBox, ComboBox, DateTimePicker).
 - **Grilla Dinámica:** Agregar múltiples actividades técnicas (desglosando la descripción de la tarea y su duración en minutos) de forma temporal dentro de un **DataGridView**.
 - **Botón de Acción Transaccional:** Ejecutar el procesamiento general que extrae los datos de la pantalla y despacha el bloque masivo hacia la capa de control.
 - **Retroalimentación Visual:** Desplegar cuadros de diálogo (MessageBox) claros y oportunos que confirmen el éxito del almacenamiento o reporten los códigos de error detectados durante la operación.

5 Entregables

El archivo ZIP deberá contener:

- Código fuente completo en C++/CLI, estructurado en capas (Modelo, Persistencia, Servicio o Controlador, Vista), con comentarios apropiados.
- Capturas de pantalla que evidencien:
 - El formulario funcionando: ingreso de mantenimiento, ingreso de datos y ejecución.
 - La ejecución exitosa del registro del mantenimiento.

6 Rúbrica de Evaluación

Puntaje Máximo: 20 puntos

Criterio	Excelente (100%)	Bueno (80%)	Regular (60%)	Deficiente (40%)	No Cumple (0%)
Modelado de la clase (4 pts)	Clases completas, bien diseñadas, relación uno-a-muchos correcta.	Ligeras omisiones.	Incompleto o con errores.	Diseño deficiente.	No implementado.
Persistencia – transacción y SPs (6 pts)	Código robusto y transaccional.	Funciona con detalles menores.	Funciona parcialmente.	Múltiples errores.	No implementado.
Controlador – integración (4 pts)	Flujo correcto y validado.	Pequeños errores.	Parcial.	Flujo deficiente.	No implementado.
Formulario – Presentación (4 pts)	Completo, funcional y validado.	Funciona con fallas menores.	Funciones básicas incompletas.	Funcionalidad deficiente.	No implementado.
Documentación y capturas (2 pts)	Completo, claro y ordenado.	Omisiones leves.	Incompleto.	Poco claro.	No entregado.